

[Ville Voutilainen](#)
[Thomas Köppe](#)
[Andrew Sutton](#)
[Herb Sutter](#)
[Gabriel Dos Reis](#)
[Bjarne Stroustrup](#)
[Jason Merrill](#)
[Hubert Tong](#)
[Eric Niebler](#)
[Casey Carter](#)
[Tom Honermann](#)
[Erich Keane](#)
[Walter E. Brown](#)
[Michael Spertus](#)

2018-10-08

Yet another approach for constrained declarations

Abstract

We propose a short syntax for the constrained declaration of function parameters, function return types and variables. The new syntax is a “constrained `auto`”, e.g. `void sort(Sortable auto& c);`.

Contents

- [1. Revision history](#)
- [2. Proposal summary](#)
- [3. Part 1: “Constrained `auto`”](#)
- [4. Part 2: Relaxed “constrained `auto`”](#)
- [5. Part 3: Meaning of “`template <Concept T>`”](#)
- [6. Part 4: Meaning of “`template <Concept... T>`” and its friends](#)
- [7. Part 5: Meaning of “`-> Concept auto`” and its friends](#)

Revision history

- This document:
 - Added discussion in Part 4 on variadic concepts.
 - Added discussion in Parts 2 and 5 on *return-type-requirements*.
- P1141R0: Initial proposal.

Proposal summary

This paper proposes three things:

1. A syntax for constrained declarations that is practically a “constrained `auto`”; the principle being “wherever `auto` goes, a `Constraint auto` can also (non-recursively) go”. The semantics are to deduce like `auto` and additionally check a constraint. In a nutshell,

```
void f(Sortable auto x);
Sortable auto f();      // #1
Sortable auto x = f();  // #2
template <Sortable auto N> void f();
```

and all combined:

```
template <Sortable auto N> Sortable auto f(Sortable auto x)
{
```

```
    Sortable auto y = init;
}
```

An unconstrained version of that is:

```
template <auto N> auto f(auto x)
{
    auto y = init;
}
```

So, this proposal includes `auto`-typed parameters for functions, which we already allow for lambdas.

2. An additional relaxation where the `auto` is optional for the cases #1 and #2 illustrated above:

```
Sortable f();
Sortable x = f();
```

3. Simplifying (and thus restricting) the rules in [\[temp.param\]/10](#), so that `template <Sortable S>` always means that `S` is a type parameter, and `template <Sortable auto S>` always means that `S` is a non-type parameter. Template template-parameters are no longer supported in this short form. Moreover, `Sortable` is restricted to be a concept that takes a type parameter or type parameter pack; non-type and template concepts are no longer supported in this short form.

`Sortable` is a “type concept” in all the examples of this summary.

This paper specifically does *not* propose

- any new lead-in syntax for templates, or
- a new syntax for introducing names for placeholder types, or
- a shortcut syntax for applying multiple constraints to a placeholder type.

The idea of this approach is to provide a syntax that

- works for constrained function parameters, constrained return types, constrained variables, and type-constrained non-type template parameters;
- avoids inventing many adventurous new things;
- in particular, avoids inventing new type sigils;
- does not clash with explicit template instantiations; and
- is compatible with what we already have in polymorphic lambdas, and makes functions uniform with them.

Part 1: “Constrained auto”

The approach proposed here borrows a subset of [P0807R0 An Adjective Syntax for Concepts](#). The idea is that we don’t try to come up with a notation that does everything that P0807 does; in particular, there is no proposal for a new syntax to introduce a type name.

Function templates

The approach is simple: allow `auto` parameters to produce function templates (as they produce polymorphic lambdas), and allow the `auto` to be preceded by a concept name. In every case, such a parameter is a deduced parameter, and we can see which parameters are deduced and which ones are not:

```
[](auto a, auto& b, const auto& c, auto&& d) {...}; // unconstrained
[](Constraint auto a, Constraint auto& b, const Constraint auto& c, Constraint auto&& d) {...}; // constrain

void f1(auto a, auto& b, const auto& c, auto&& d) {...}; // unconstrained
void f2(Constraint auto a, Constraint auto& b, const Constraint auto& c, Constraint auto&& d) {...}; // constrained

[](Constraint auto&& a, SomethingElse&& b) {...}; // a constrained deduced forwarding reference and a concrete parameter
void f3(Constraint auto&& a, SomethingElse&& b) {...}; // a constrained deduced forwarding reference and a concrete parameter
```

The appearance of `auto` (including `Constraint auto`) in a parameter list tells us that we are dealing with a function template. For each parameter, we know whether it is deduced or not. We can tell apart concepts from types: concepts precede `auto`, types do not.

Return types and variable declarations

Constrained return types work the same way:

```

auto f4();                // unconstrained, deduced.

Constraint auto f5();      // constrained, deduced.

Whatever f6();            // See part 2. If Whatever is a type, not deduced.
                          // If Whatever is a concept, constrained and deduced.

```

Note that **f4**, **f5** and **f6** are not templates (whereas the previous **f1**, **f2** and **f3** are templates). Here, there is no mention of **auto** in the parameter list. Users have the choice of adopting a style where it is explicit as to whether the return type is deduced.

Constrained types for variables work the same way:

```

auto x1 = f1();           // unconstrained, deduced.

Constraint auto x2 = f2(); // constrained, deduced.

Whatever x3 = f3();       // See part 2. If Whatever is a type, not deduced.
                          // If Whatever is a concept, constrained and deduced.

```

Again, users can make it so that it is easy to see when deduction occurs.

Since non-type template parameters can be deduced via **auto** (as in `template <auto N> void f();`), we also allow a constraint there:

```

template <Constraint auto N> void f7();

```

Note, however, that this can only be a type constraint; non-type concepts (including auto concepts) are not allowed in this form.

Other uses of auto

In concert with the general approach that “**Constraint auto** goes wherever **auto** goes”, new-expressions and conversion operators work:

```

auto alloc_next() { return new Sortable auto(this->next_val()); }

operator Sortable auto() { }

```

A “**Constraint auto**” cannot be used to indicate that a function declarator has a trailing return type:

```

Constraint auto f() -> auto; // ill-formed; shall be the single type-specifier auto

```

decltype(auto) can also be constrained:

```

auto f() -> Constraint decltype(auto);
Constraint decltype(auto) x = f();

```

Structured bindings do deduce **auto** in some cases; however, the **auto** is deduced from the whole (and not from the individual components). It is somewhat doubtful that applying the constraint to the whole, as opposed to (for example) applying separately to each component, is the correct semantic. Therefore, we propose to defer enabling the application of constraints to structured bindings to separate papers.

General rules

The constraint applies directly to the deduced type. It does not apply to the possibly cv-qualified type described by the type specifiers, nor does it apply to the type declared for the variable:

```

const Assignable<int> auto&& c = *static_cast<int *>(p); // Assignable<int &, int>

```

Naturally, if the deduced type is cv-qualified (or a reference), the constraint applies to that type.

To keep things simple, an **auto** (or **decltype(auto)**) being constrained is always immediately preceded by the constraint. So, cv-qualifiers and concept-identifiers cannot be freely mixed:

```

const Constraint auto x = foo(); // ok
Constraint const auto x = foo(); // ill-formed
Constraint auto const y = foo(); // ok

```

We propose only the ability to apply one single constraint for a parameter, return type, or non-type template parameter. Any proposal to consider multiple constraints should happen separately after C++20.

Partial concept identifiers also work. Given a concept `template <typename T, typename... Args> concept Constructible = /* ... */;`, we can say:

```
void f(Constructible<int> auto x);    // Constructible<decltype(x), int> is satisfied

Constructible<int> auto f();

Constructible<int> auto x = f();

template <Constructible<int> auto N> void f();
```

Part 2: Relaxed “constrained auto”

In the return type of a function declaration, we can leave out the `auto`. So, in addition to

```
Constraint auto f1();
```

we can write

```
Constraint f2();
```

Neither `f1` nor `f2` are templates. It seems fairly reasonable to allow omitting the `auto`, but that is intended to be a relaxation of the general rule, not a replacement for `Constraint auto`.

Trailing return types in function declarations behave the same way:

```
auto f1() -> Constraint; // okay
```

Trailing return types are also used for *return-type-requirements*; however, `auto` does not (currently) go in such a context, whereas constraints *do*. A minimal change we can make as part of this proposal is to take `auto` as being implicitly modified by a least constrained concept where a constraint is currently allowed in a *return-type-requirement*. At the same time, `Constraint auto` would also be allowed. Thus, a plain `Constraint` in such a position can be seen as a case of relaxed “constrained `auto`”. The interaction with [P1084R0 Today’s return-type-requirements Are Insufficient](#) is discussed later in this paper.

For example:

```
requires(T t) {
    {t} -> const auto*; // t needs to be a pointer to const
}

requires(T t) {
    {t} -> Constraint auto; // t needs to be of a type that satisfies Constraint
}
```

In variable declarations, omitting the `auto` also seems reasonable:

```
Constraint x = f2();
```

Note, in particular, that we already have a syntax that does (partial) deduction but doesn’t make that explicit in the syntax:

```
std::tuple x = foo();
```

The variable case in particular seems reasonable, considering the already existing deduction syntaxes that don’t call attention to deduction. The user always has a choice to use a more explicit syntax. The return type case might well have a weaker rationale for being allowed. It should be noted, though, that this relaxation in general was present in the TS; this paper is merely not proposing it for parameters.

Certain disambiguation details need to be handled:

```
bool b(Constructible<int> && bar());    // variable definition

void foo() {
    Constructible<int> * f2();           // disambiguation/type-name interpretation rule required
    Constructible<int> * selector = f2(); // (same?) disambiguation/type-name interpretation rule require
}
```

Note that the variable definition is unambiguously a variable definition. This proposal proposes no function declaration syntax that would clash with it; in particular, `Constructible<int>` is not considered a *type-name* that is short for `Constructible<int>`

`auto` except in limited contexts. It's merely something to be aware of, and a demonstration of how the presence of `auto` avoids ambiguity. For each remaining case above, `Constructible<int>` *does* appear within such a limited context when parsing the statements as prospective declaration statements; we propose to generalize the rule for disambiguating in favor of declaration statements to cover this case.

Part 3: Meaning of “`template <Concept T>`”

In [\[temp.param\]/10](#) we have:

A *constrained-parameter* declares a template parameter whose kind (type, non-type, template) and type match that of the prototype parameter (17.6.8) of the concept designated by the *qualified-concept-name* in the *constrained-parameter*. Let *X* be the prototype parameter of the designated concept. The declared template parameter is determined by the kind of *X* (type, non-type, template) and the optional ellipsis in the *constrained-parameter* as follows.

- If *X* is a type *template-parameter*, the declared parameter is a type *template-parameter*.
- If *X* is a non-type *template-parameter*, the declared parameter is a non-type *template-parameter* having the same type as *X*.
- If *X* is a template *template-parameter*, the declared parameter is a template *template-parameter* having the same *template-parameter-list* as *X*, excluding default template arguments.
- If the *qualified-concept-name* is followed by an ellipsis, then the declared parameter is a template parameter pack (17.6.3).

[Example:

```
template<typename T> concept C1 = true;
template<template<typename> class X> concept C2 = true;
template<int N> concept C3 = true;
template<typename... Ts> concept C4 = true;
template<char... Cs> concept C5 = true;

template<C1 T> void f1();      // OK, T is a type template-parameter
template<C2 X> void f2();      // OK, X is a template with one type-parameter
template<C3 N> void f3();      // OK, N has type int
template<C4... Ts> void f4();   // OK, Ts is a template parameter pack of types
template<C4 T> void f5();      // OK, T is a type template-parameter
template<C5... Cs> void f6();   // OK, Cs is a template parameter pack of chars
```

—end example]

Does that seem like a mouthful?

That's because it is. In `template <Constraint T>`, the kind of *T* depends on the kind of the prototype parameter of *Constraint*.

We instead propose that, for such a constrained-parameter syntax:

- *T* should always be a type, and
- *Constraint* would always need to be a concept that has a corresponding type parameter or type parameter pack.

To be clear, we are not proposing that concepts in general should not have non-type or template template parameters. We are merely proposing for it to be the case that the constrained parameter shortcut is not provided for concepts with such prototype parameters; such concepts would need to be used with a *requires-clause*. The constrained parameter syntax should mean just one thing. Note that the same syntax `template <A T>` is still a non-type parameter when *A* is a type name rather than a concept. We are willing to tolerate this small potential for ambiguity.

The rationale for this part is as follows:

1. It seems desirable to have the constrained template parameter syntax.
2. It would be nice if that syntax covered the most common case.
3. It would further be nice if that syntax covered *only* the most common case.
4. The other cases are expected to be so rare that there's no need to provide a shortcut for them, and they are certainly rare enough that they shouldn't use the same syntax.

So, to clarify:

- `template <MyIntTypeDef N>` means a non-type parameter, like it always did.

- `template <ConceptName T>` means a type parameter constrained by `ConceptName`, and the prototype parameter of `ConceptName` needs to be a type parameter or a type parameter pack.
- `template <auto N>` means a non-type parameter with a deduced type.
- `template <ConceptName auto N>` means a non-type parameter with a deduced type constrained by `ConceptName`, and the prototype parameter of `ConceptName` needs to be a type parameter or a type parameter pack.

Other use cases can be done with *requires-clauses*.

Part 4: Meaning of “`template <Concept... T>`” and its friends

In [\[temp.param\]/11](#) we have:

```
template<C2... T> struct s3; // associates C2<T...>
```

This seems to be doing an unexpected thing, which is having the constraint apply to more than one type in a pack at a time. We propose that, regardless of whether the prototype parameter of the named concept is a pack:

- For a simple pack of constrained types, the concept mentioned is applied, as a unary concept, to each type in the pack in turn.
- For a pack of constrained types that use *partial-concept-ids*, the concept mentioned is applied, as an n-ary concept whose arity is unaffected by the size of the pack, *individually* to each type in the pack in turn.

In other words,

- `template <ConceptName... T> void f(T...);` means a variadic function template where each type in the pack `T` needs to satisfy `ConceptName` as a unary concept, applied as `ConceptName<Tn>`.
- Similarly, `void f(ConceptName auto... T);` means exactly the same thing.
- `template <ConceptName<int>... U> void f(U...);` means a variadic function template where each type in the pack `U` needs to satisfy `ConceptName` as a binary concept, applied as `ConceptName<Un, int>`.
- Similarly, `void f(ConceptName<int> auto... U);` means exactly the same thing.
- `template <ConceptName<0u, void, wchar_t>... U> void f(U...);` means a variadic function template where each type in the pack `U` needs to satisfy `ConceptName` as a n-ary concept, applied as `ConceptName<Un, 0u, void, wchar_t>`.

Part 5: Meaning of “`-> Concept auto`” and its friends

`Constraint auto`, by virtue of being allowed where `auto` may appear, may now appear (at least syntactically) in a trailing return type. Following the general rule that we deduce like `auto` (and additionally check the constraint), a meaning is given for “`-> Concept auto`” when it appears as the trailing return type in a function definition. The meaning being that we get plain return type deduction (as with `auto`) plus a check of the constraint.

In [Part 2](#), we note that trailing return types are also used for *return-type-requirements* and we propose for a meaning to be given for use of plain `auto` in such a context. Further building along that direction, the form of *return-type-requirement* involving a concept to be satisfied becomes merely a generalization of *trailing-return-types* with placeholder types. The deduction as described in Part 2 is based on the status quo of the post-Rapperswil working draft, and it is similar to that of the TS and to the deduction that occurs for return type deduction.

P1084R0 explains that the existing deduction rules associated with *return-type-requirements* is lacking in terms of expressing certain desired semantics. As a solution, it proposed making `-> Constraint` more of a special case in the context of a *return-type-requirement*. Expressed in terms of deduction, the semantics proposed by P1084R0 is more akin to deduction for a theoretical `decltype(auto)` than to deduction for `auto`. Which is to say that the semantics for

```
-> Constraint
```

as proposed in P1084R0 is incongruous with how we would describe the semantics of

```
-> Constraint auto
```

in the context of this paper.

This paper raises the possibility of expressing the semantics requested in P1084R0 in a different manner. `{(E)} -> Constraint decltype(auto);` produces the effect that P1084R0 requests for `{E} -> Constraint`; in terms of applying `Constraint` with `decltype(E)` as the argument to its prototype parameter. There is a further consideration as to whether convertibility, as

opposed to type identity, should be enforced; in particular, P1084R0 leaves the possibility that `{A()} -> std::Same<A>` would be satisfied, but `{A()} -> A` would not be.

Three different kinds of convertibility requirement have been applied in the various iterations of *return-type-requirement* thus far. With WD signifying [N4762](#), TS signifying [N4674](#), and *return* representing return type deduction (whether under the TS or under this proposal); the table below summarizes:

	Concrete type				Placeholder type			
	WD	TS	return	P1084	WD	TS	return	P1084
Deduction only					X			X
Implicit conversion sequence						X		
Well-formed copy initialization	X	X	X	X			X	

The deduction-only situation with the WD may well be a wording defect, since `{0} -> std::Same<int>&` would surprisingly not fail. Similarly, for the TS, the difference between requiring an implicit conversion sequence and requiring a well-formed copy initialization is somewhat subtle. Nevertheless, the treatment for concrete types differ from that of types involving placeholders for every iteration of *return-type-requirement* thus far (given that return type deduction is present for comparison only). This seems undesirable.

The WD situation shows that some sort of requirement would be needed since deduction is not sufficient. A convertibility requirement is an option; however, P1084 does not impose a convertibility requirement for the placeholder case. Under P1084, the placeholder case would trivially pass a value category-aware type identity check. Requiring such a check of `decltype(E)` being the same as the type in the *return-type-requirement* (after substitution of the deduced placeholders, if any) is also an option. P1084 proposes a generalization of a same-type constraint, applying a change only to the placeholder case; this latter option is an application of a generalized same-type constraint on the non-placeholder case (as well as to placeholder cases that P1084 drops).

For a parenthesized lvalue of type `int`, the following table describes the value category-aware type identity check:

	Type deduced/passed to prototype parameter	Type to match	Type matches
auto	int	int	no
C	int	int	no
C&	int	int&	yes
C&& (forwarding reference)	int&	int&	yes
C decltype(auto)	int&	int&	yes
const C&	int	const int&	no
int	N/A	int	no
int&	N/A	int&	yes
int&&	N/A	int&&	no

In brief, `auto` means “is a prvalue”, `const C&` means “is an lvalue of `const T` requiring `C<T>`”, `int&&` means “is an `int` xvalue”, and `C decltype(auto)` has the semantics requested in P1084 for all parenthesized expressions.